

Conception et Implémentation d'une Base de Données Distribuée

pour une Plateforme e-Commerce B2B

Oracle XE 21c · Docker · PL/SQL · Triggers multi-maîtres

Hajar Elhilali, Yasmine Koulam

Cycle Ingénieur – Génie Informatique – 2^{ème} Année

Université Hassan 1^{er} – Faculté des Sciences et Techniques de Settat

Année universitaire 2025 – 2026

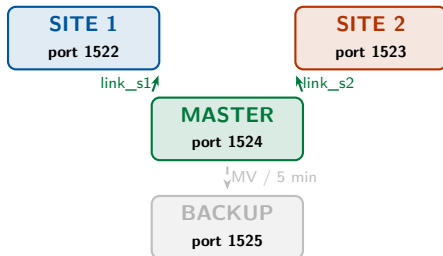
Problématique

Comment distribuer la table `LigneCommandes` sur plusieurs serveurs tout en garantissant la **cohérence** et la **transparence** pour les applications ?

Solution

- **Fragmentation horizontale** de `LigneCommandes`
- **2 scénarios** de distribution
- Synchronisation par **triggers PL/SQL**
- Identifiants uniques : **séquences modulo 3**
- Sauvegarde : **vues matérialisées**

Objectif : distribuer EShop B2B sur **4 nœuds Oracle XE 21c** conteneurisés (Docker), avec routage automatique des données.



Nœud	Port	Rôle
Master	1524	Base globale + seed
Site 1	1522	Fragment R_1
Site 2	1523	Fragment R_2
Backup	1525	Vues matérialisées

- **DB Links privés** + TNS partagé
- Compte `eshop_link` à moindre privilège
- Sites démarrent après le Master

Deux Scénarios de Fragmentation Horizontale

Scénario 1 – Dérivée

- $R_1 \rightarrow$ **Site 1** : cat=50 et qté>100
- $R_2 \rightarrow$ **Site 2** : cat=35 et qté>50

2 critères (catégorie + quantité)

Partition **partielle** (cat 10 reste sur Master)

Scénario 2 – Primaire

- $R_1 \rightarrow$ **Site 1** : qté $\geq$ 100
- $R_2 \rightarrow$ **Site 2** : qté < 100

1 seul critère (quantité)

Partition **totale** : $R_1 \cap R_2 = \emptyset$

Les deux scénarios routent automatiquement chaque opération vers le bon fragment via des **triggers PL/SQL sur le Master** – **transparence totale** pour les applications.

SYC_INSERT_LIGNE (sur le Master)

```
-- Apres INSERT, le Master analyse la
-- categorie du produit et route la ligne
SELECT idcateg INTO v_cat FROM produits
WHERE idproduit = :NEW.idproduit;

IF (v_cat = 50 AND :NEW.quantite > 100) THEN
    INSERTligne@link_site1(...);    -- Site 1
ELSIF (v_cat = 35 AND :NEW.quantite > 50)
    THEN
    INSERTligne@link_site2(...);    -- Site 2
END IF;
```

Principe

- L'application écrit **uniquement** sur le Master
- Le trigger propage vers le bon site
- INSERT / UPDATE / DELETE : un trigger chacun

Cohérence des clés étrangères

Un trigger **BEFORE INSERT** rapatrie produit, commande et client depuis le Master avant l'écriture.

SYC_UPDATE_LIGNE – Scénario 2

```
-- La ligne etait sur Site 1 (qte>=100)
IF (OQ >= 100) THEN
  IF (NQ >= 100) THEN
    updateligne@link_site1(...); --
    reste
  ELSE --
    migration
    DELETEligne@link_site1(...);
    INSERTligne@link_site2(...);
  END IF;
-- (cas symetrique pour Site 2)
END IF;
```

4 cas selon le seuil 100

$\geq 100 \rightarrow \geq 100$	reste Site 1
$\geq 100 \rightarrow < 100$	Site 1 $\rightarrow$ Site 2
$< 100 \rightarrow \geq 100$	Site 2 $\rightarrow$ Site 1
$< 100 \rightarrow < 100$	reste Site 2

Cross-site

Modifier depuis Site 2 une donnée stockée sur Site 1 fonctionne : le routage passe toujours par le Master.

Anti-boucle : pkg_synchro

```
PACKAGE pkg_synchro AS
    is_replicating BOOLEAN := FALSE;
END;
-- Debut de chaque trigger :
IF pkg_synchro.is_replicating
    THEN RETURN; END IF;
```

Compound Trigger

- **AFTER EACH ROW** : collecte les IDs
- **AFTER STATEMENT** : DELETE en lot
- Évite l'erreur **ORA-04091** (table en mutation)

Journalisation

PRAGMA AUTONOMOUS_TRANSACTION : les erreurs sont enregistrées dans `sync_errors` indépendamment du ROLLBACK.

Problème

Génération autonome des IDs sur chaque nœud $\Rightarrow$ risque de **collisions** de clés primaires.



Solution : 3 voies disjointes

Nœud	$\text{mod}(\text{ID}, 3)$	Valeurs
Master	1	1, 4, 7...
Site 1	2	2, 5, 8...
Site 2	0	3, 6, 9...

Garantie

Les 3 ensembles sont **strictement disjoints** : aucune collision possible, à vie.

Q5 – Index couvrant

```
CREATE INDEX ix_cmd_date_client  
ON commandes(datecommande, idclient);
```

Accès COMMANDES	Avant	Après
Plan	FULL SCAN	INDEX RANGE
Coût	7	5
Gain		-28%

Q6 – CA distribué (UNION ALL)

```
SELECT idcateg, SUM(ca) AS CA_2026  
FROM (  
  SELECT ... FROM ...@link_site1  
  UNION ALL      -- R1 inter R2 = vide  
  SELECT ... FROM ...@link_site2  
) GROUP BY idcateg;
```

Pourquoi UNION ALL ?

Fragments disjoints $\Rightarrow$ pas de doublons. On évite le tri inutile de UNION.

Réalisations

- **2 scénarios** de fragmentation opérationnels
- Réplication par **triggers** + **pkg_synchro**
- **Migration automatique** (seuil de routage)
- Séquences **modulo 3** : zéro collision
- Sauvegarde par **vues matérialisées**
- Optimisation Q5 (−28%) et Q6 distribuée

Perspectives

- Fragmentation verticale
- Réplication asynchrone
- Load balancing inter-sites

Merci pour votre attention – Questions ?